



LangChain

Building Applications with Large Language Models

Created by: Het Rank

OVERVIEW

Agenda

01

Introduction

What LangChain is and why it exists

02

Requirements

Setting up your environment

03

Retrieval Augmented Generation (RAG)

Grounding LLMs in real data

04

LangChain Components & Use Cases

Chains, agents, memory, and applications

05

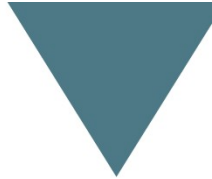
Demo

Seeing it in action

SECTION 1

Introduction

Getting oriented



INTRODUCTION

What is LangChain

1

Open-source framework

Built for developing applications powered by large language models (LLMs).

2

Tools & APIs

Provides the building blocks to create chatbots and virtual agents.

3

Better outputs

Improves accuracy and relevance of LLM responses through structured prompting and retrieval workflows.

4

Multi-language

Available as libraries in Python, JavaScript, and TypeScript.

SECTION 2

Requirements

Before you build

REQUIREMENTS

How to Install & Set Up

System Setup

- 1 Install Python (python.org)
- 2 Use an editor — VS Code or Jupyter
- 3 Install the Python extension in VS Code
- 4 Open a project folder for the app

Install Commands

```
pip install langchain
```

```
pip install langchain-community
```

```
pip install openai
```

Tip: if you see a “module not found” error while running your code, install the missing package with pip.

SECTION 3

Retrieval Augmented Generation

Grounding LLMs in real data

What is RAG?

Retrieval Augmented Generation (RAG) enhances language models by combining them with external knowledge bases. It relies on two components: a document retriever and an LLM.

How retrieval works

The retriever finds relevant information from a large document corpus using semantic search (context-aware, BERT-like models) or similarity search (vector-based, e.g. cosine similarity).

Why RAG matters

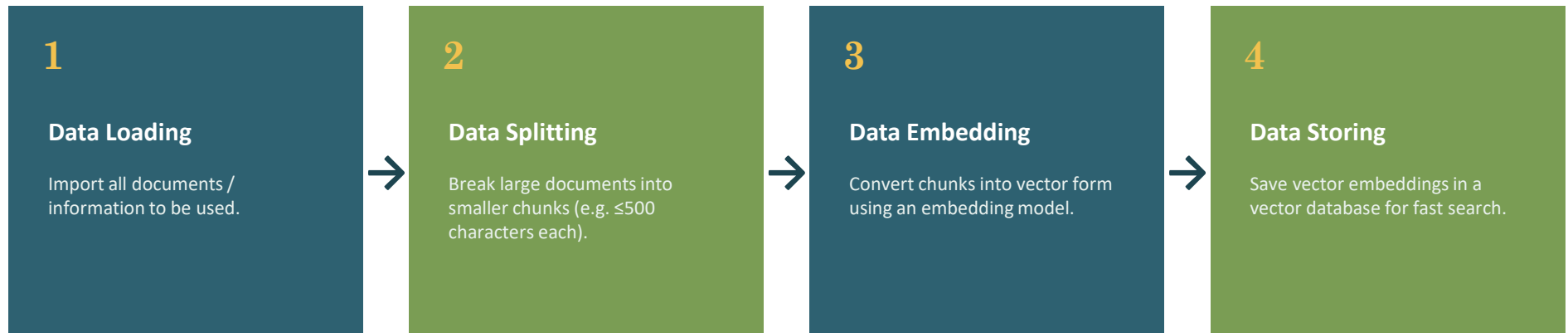
LLMs alone can hallucinate, rely on outdated information, and reason in a black-box way. RAG grounds responses in domain-specific data to solve these issues.

LLM Pain Points RAG Solves

- ! Hallucinations
- ! Outdated knowledge
- ! Black-box reasoning

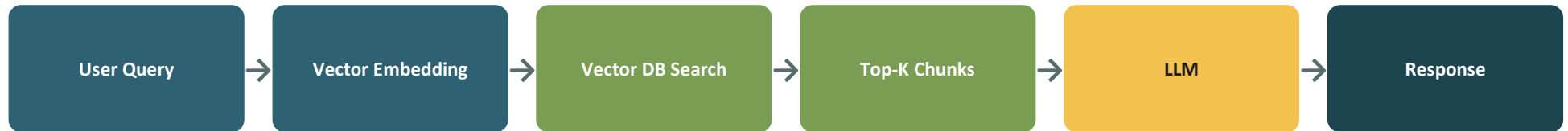
Solved by combining an LLM with a retriever over your own documents.

Data Indexing



Documents → Chunks → Vector Embeddings → Vector Database

Retrieval & Generation



1. Retrieval

The user's input is transformed into a query vector using the same embedding model from Data Indexing, then matched against the vector database to find the most similar chunks.

2. Generation

The LLM combines the user's question with the retrieved chunks to generate a grounded, context-aware response.

SECTION 4

LangChain Components & Use Cases

Beyond the basics

Building Blocks & Applications

Core Components

1

Chains

Sequences of calls linking prompts, models, and logic together.

2

Agents

LLMs that dynamically choose which tools or actions to take.

3

Memory

Retains conversation context across multiple turns.

4

Vector Store Integrations

Connects to stores like FAISS, Pinecone, and Chroma.

Common Use Cases

- Chatbots & virtual assistants
- Question-answering over private documents
- Summarization tools
- Autonomous agents for task automation
- Data extraction pipelines

SECTION 5

Demo

Let's see it in action



Thank You

Questions & Discussion

Created by: Het Rank